

EXPERIMENT 9

Student Name: Husanpreet Kaur

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70138

Section/Group: 24AIT_KRG-G2

Date: 31-08-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To understand and implement PostgreSQL stored procedures, conditional stock management, sales processing, foreign key constraints, CASCADE operations, and table management using PL/pgSQL, while maintaining data consistency and integrity between related tables.

Question 1: An organization maintains employee details in a table named `employee_info`. The table contains the employee ID, name, gender, salary, and city.

Create a PostgreSQL stored procedure that accepts an employee's gender as an IN parameter and calculates the total number of employees belonging to that gender.

The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE.

Solution: CREATE TABLE `employee_info` (

 id SERIAL PRIMARY KEY,

 name VARCHAR(50) NOT NULL,

 gender VARCHAR(10) NOT NULL,

 salary NUMERIC(10,2) NOT NULL,

 city VARCHAR(50) NOT NULL

);

INSERT INTO `employee_info` (name, gender, salary, city)

EXPERIMENT 9

VALUES

('Alok', 'Male', 50000.00, 'Delhi'),
('Priya', 'Male', 60000.00, 'Mumbai'),
('Rajesh', 'Female', 45000.00, 'Bangalore'),
('Sneha', 'Male', 55000.00, 'Chennai'),
('Anil', 'Male', 52000.00, 'Hyderabad'),
('Sunita', 'Female', 48000.00, 'Kolkata'),
('Vijay', 'Male', 47000.00, 'Pune'),
('Ritu', 'Male', 62000.00, 'Ahmedabad'),
('Amit', 'Female', 51000.00, 'Jaipur');

```
CREATE OR REPLACE PROCEDURE count_employees_by_gender(  
    IN p_gender VARCHAR,  
    OUT employee_count INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Count employees belonging to the given gender  
    SELECT COUNT(*)  
    INTO employee_count  
    FROM employee_info  
    WHERE gender = p_gender;
```

EXPERIMENT 9

-- Display the result as a message

RAISE NOTICE 'Total number of % employees: %',

p_gender, employee_count;

END;

\$\$;

CALL count_employees_by_gender('Male', NULL);

CALL count_employees_by_gender('Female', NULL);

OUTPUT:

Data Output	Messages	Notifications
SQL Showing rows: 1 to 1 of 1 Page No: 1		
	employee_count integer	
1	12	

Data Output	Messages	Notifications
SQL Showing rows: 1 to 1 of 1 Page No: 1		
	employee_count integer	
1	6	

Question 2: An e-commerce company maintains product stock information in a products table and completed order information in a sales table. Create a PostgreSQL stored procedure named pr_buy_products that accepts the product name and required quantity as input parameters.

The procedure should check whether the requested product exists and whether sufficient stock is available. If sufficient stock is available, the procedure should:

1. Retrieve the product code and price.
2. Calculate the total sale amount using:

Sale Price = Product Price × Quantity Ordered

3. Insert a new order record into the sales table.

EXPERIMENT 9

4. Reduce the quantity_remaining in the products table.
5. Increase the quantity_sold by the purchased quantity.
6. Display a success notification.

If sufficient stock is not available, the procedure should not modify any data and should display an appropriate insufficient quantity notification.

Create the required tables, insert sample data, create the stored procedure pr_buy_products, and execute it to verify the result.

Solution: CREATE TABLE products (

```
product_code VARCHAR(10) PRIMARY KEY,  
product_name VARCHAR(100) NOT NULL,  
price NUMERIC(10,2) NOT NULL,  
quantity_remaining INT NOT NULL,  
quantity_sold INT DEFAULT 0
```

);

CREATE TABLE sales (

```
order_id SERIAL PRIMARY KEY,  
order_date DATE NOT NULL,  
product_code VARCHAR(10) NOT NULL,  
quantity_ordered INT NOT NULL,  
sale_price NUMERIC(10,2) NOT NULL,  
FOREIGN KEY (product_code) REFERENCES products(product_code)
```

);

EXPERIMENT 9

```
INSERT INTO products (product_code, product_name, price, quantity_remaining,  
quantity_sold)
```

```
VALUES
```

```
('P001', 'iPHONE 13 PRO MAX', 109999.00, 10, 0),  
( 'P002', 'Samsung Galaxy S23 Ultra', 99999.00, 8, 0),  
( 'P003', 'iPAD AIR', 55999.00, 5, 0),  
( 'P004', 'MacBook Pro 14"', 189999.00, 3, 0),  
( 'P005', 'Sony WH-1000XM5 Headphones', 29999.00, 15, 0);
```

```
INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
```

```
VALUES
```

```
('2025-09-15', 'P001', 1, 109999.00),  
( '2025-09-16', 'P002', 2, 199998.00),  
( '2025-09-17', 'P003', 1, 55999.00),  
( '2025-09-18', 'P005', 2, 59998.00),  
( '2025-09-19', 'P001', 1, 109999.00);
```

```
SELECT * FROM PRODUCTS;
```

```
SELECT * FROM SALES;
```

```
CREATE OR REPLACE PROCEDURE pr_buy_products(  
    IN p_product_name VARCHAR,  
    IN p_quantity INT  
)
```

EXPERIMENT 9

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_product_code VARCHAR(10);
    v_price NUMERIC(10,2);
    v_count INT;
    v_total NUMERIC(10,2);
BEGIN

    -- Check whether product exists and sufficient stock is available
    SELECT COUNT(*)
    INTO v_count
    FROM products
    WHERE product_name = p_product_name
        AND quantity_remaining >= p_quantity;

    -- If sufficient stock is available
    IF v_count > 0 THEN

        -- Retrieve product code and price
        SELECT product_code, price
        INTO v_product_code, v_price
        FROM products
        WHERE product_name = p_product_name;
```

EXPERIMENT 9

-- Calculate total sale amount

v_total := v_price * p_quantity;

-- Insert new order into sales table

INSERT INTO sales (

order_date,

product_code,

quantity_ordered,

sale_price

)

VALUES (

CURRENT_DATE,

v_product_code,

p_quantity,

v_total

);

-- Reduce remaining stock and increase quantity sold

UPDATE products

SET quantity_remaining = quantity_remaining - p_quantity,

quantity_sold = quantity_sold + p_quantity

WHERE product_code = v_product_code;

EXPERIMENT 9

-- Success message

RAISE NOTICE

'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.',

p_quantity, p_product_name;

ELSE

-- No data modification

RAISE NOTICE

'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s) of %.',

p_quantity, p_product_name;

END IF;

END;

\$\$;

CALL pr_buy_products('MacBook Pro 14"', 1);

CALL pr_buy_products('MacBook Pro 14"', 10);

EXPERIMENT 9

OUTPUT:

```
Data Output  Messages  Notifications
NOTICE:  PRODUCT SOLD..! Order placed successfully for 1 unit(s) of MacBook Pro 14".
CALL

Query returned successfully in 136 msec.
```

```
Data Output  Messages  Notifications
NOTICE:  INSUFFICIENT QUANTITY..! Order cannot be processed for 10 unit(s) of MacBook Pro 14".
CALL

Query returned successfully in 196 msec.
```

LEARNING OUTCOMES:

1. Understand the concept and syntax of stored procedures in PostgreSQL using PL/pgSQL.
2. Implement input parameters, conditional statements, variables, and RAISE NOTICE messages in stored procedures.
3. Apply SQL operations such as SELECT, INSERT, and UPDATE to process product purchases and manage stock efficiently.
4. Understand and implement foreign key constraints and ON DELETE CASCADE / ON UPDATE CASCADE for maintaining referential integrity.
5. Execute and verify database operations such as creating, modifying, and deleting tables while ensuring data consistency between related tables.